CONNECTOR NETWORK

The Factory & its DX Layer.
Pipeline · Auto-Sync · Transforms · Previews.

Three parts. First the full pipeline — how any source climbs to a single IR and fans out into every surface, and how messy real-world inputs are unified. Then the developer-experience layer built on top — keeping surfaces in sync, reshaping specs without touching the source, and previewing the blast radius of a change. Then the user flow: exactly what a user sees, clicks, and does at each stage — designed from how Fern, Stainless, and Speakeasy actually work, user-friendly but built for developer ergonomics.

PART I
The pipeline / factory

PART II
Auto-Sync · Transforms · Preview

PART III
User flow & prior art

GROUNDED IN
Fern · Stainless · Speakeasy

STATUS
Scope · v2.0

A single reference for the pipeline mechanics, the DX-layer scope, and the end-to-end user flow. Design decisions are informed by the public docs and engineering writing of comparable systems. Code and artifacts are illustrative.

PART I

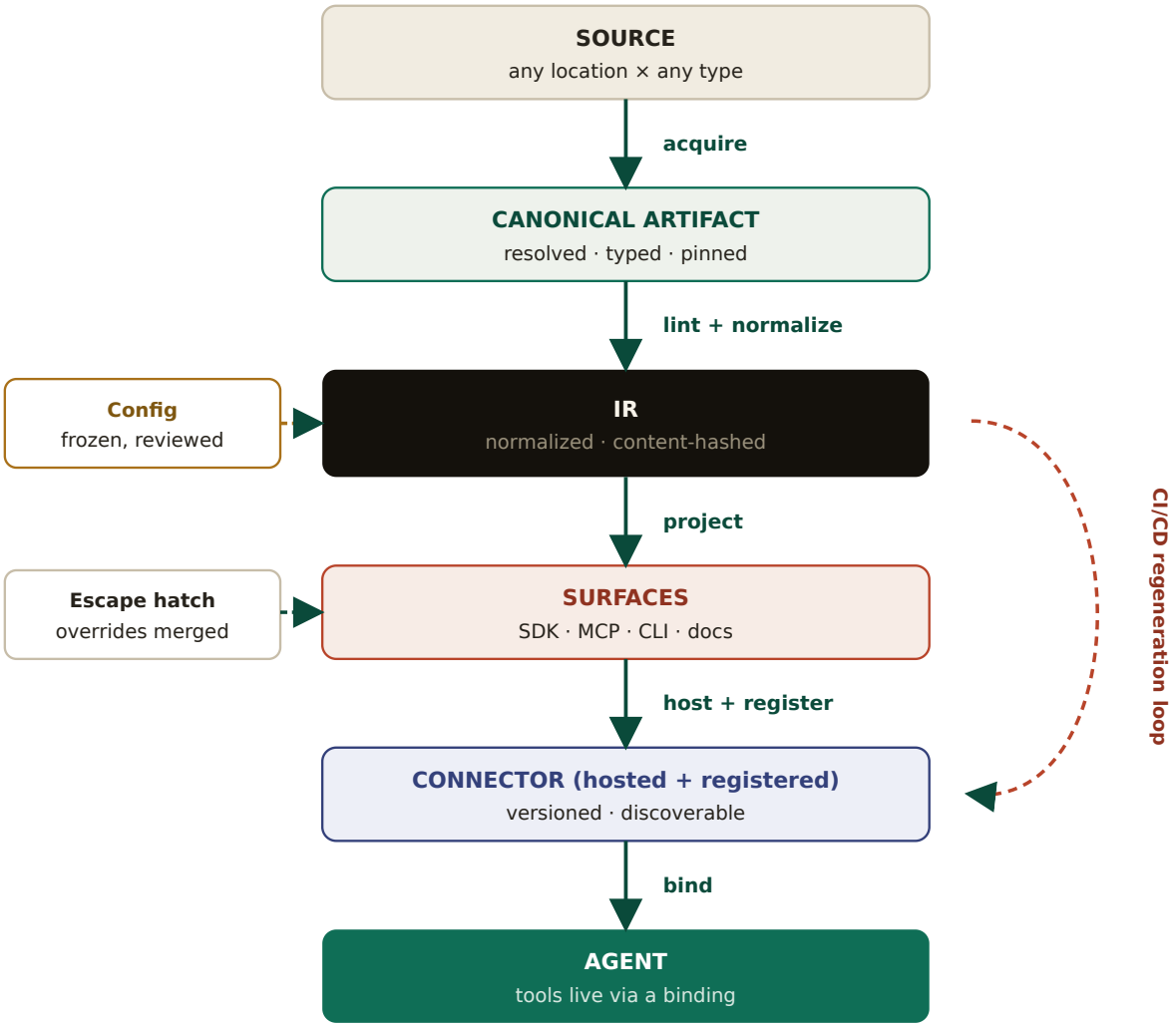
The Pipeline

The factory the DX layer extends. A source climbs once to a hashed IR and fans out into surfaces that cannot drift; this part covers the spine, how messy inputs are unified, what each stage guarantees, the live call path, and the regeneration loop that re-runs it all.

I.0 The pipeline at a glance

The pipeline is a one-way climb followed by a fan-out. Any source *climbs up* to a single normalized model — the IR — and every consumable surface is *projected down* from it. Nothing is ever converted sideways (spec→SDK directly, or SDK→MCP directly); everything routes through the IR. That single rule is what makes the whole thing deterministic, versionable, and safe to regenerate — and it is the foundation every DX capability in Part II relies on.

FIG I.1 — The full pipeline spine (one source → one IR → many surfaces → a bound agent)



Two side-inputs shape the climb: **config** (frozen human judgement) and the **escape hatch** (hand-written overrides re-applied at projection). The dashed red loop — detailed in I.4 — is what Part II's Auto-Sync runs as a standing service.

THE ONE INVARIANT EVERYTHING RESTS ON

Surfaces are **pure functions of the IR**. Given the same canonical artifact and the same frozen config, the IR hash is identical, so every surface is byte-identical. This is why connectors can be trusted and pinned, why regeneration is safe, and why Transforms and Auto-Sync in Part II can run unattended.

I.1 The worked example we'll follow

To make the stages concrete rather than abstract, the rest of Part I traces one operation belonging to a fictional producer — **Lumen**, a design-tool SaaS — from its repository all the way to a live agent.

- **Producer:** Lumen, a REST API in a *private* GitHub repo, OpenAPI 3.1 split across files.
- **The operation we trace:** create a design project — `POST /projects`.
- **The consumer:** a client running an agent that should be able to create Lumen projects on a user's behalf.

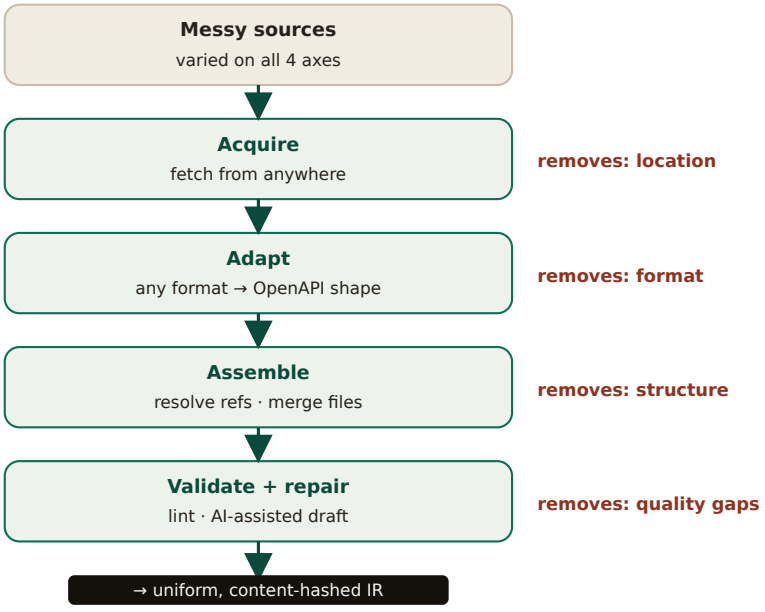
By the end, an agent will call a `create_project` tool, and we will have seen exactly what happened at every hop in between — watching the *artifact*, not the code, transform along the way.

TERM	USED THROUGHOUT THIS DOCUMENT TO MEAN
Source	What a producer brings — described by *location* (upload, Git, URL, live API) × *type* (OpenAPI, SDK, MCP server).
Canonical artifact	The single, fully-resolved, version-pinned document acquisition hands to ingestion — origin-agnostic.
IR	The normalized, projection-agnostic internal model. The hub every surface is rendered from.
Surface / projection	One consumable form of a connector (SDK, MCP, CLI, docs) — a render of the IR.
Connector	The deployable unit: all surfaces + metadata + version history + governance policy.
Binding	A live attachment of a connector's MCP surface to a specific agent, with scoped tools and brokered credentials.

I.2 Handling messy inputs → one unified format

"Accept the source" hides real work, because a source can be messy on four independent axes at once. Handling it is not one step — it is a funnel of narrowing transforms, where each stage strips away one *kind* of variation until what remains is uniform enough for the IR builder to map mechanically.

FIG I.2 — The narrowing funnel: each stage removes one axis of mess



The principle is **converge early, normalize late**: collapse the format axis fast (Adapt), so the expensive structural and semantic work is written once against one shape rather than N times against N formats.

HOW EACH STAGE SOLVES ITS AXIS

- **Acquire (location).** Authenticate, fetch (clone a tree / download / hit a discovery endpoint), and pin the exact revision. After this, downstream is origin-blind.
- **Adapt (format) — the adapter pattern.** One internal target shape (an OpenAPI-3.1-shaped *normalized spec*) and one small adapter per format, each implementing `detect()` + `adapt()`. This turns an N-formats × M-stages problem into N adapters + one downstream; a new format is a new file, not a pipeline edit. Existing SDKs and MCP servers are reverse-derived here by introspection (Python hints/Pydantic map cleanly; an MCP server's `inputSchema` maps cleanest of all).
- **Assemble (structure).** Inline every `$ref`; break circular references with a dedup table; merge multi-spec monorepos under one namespace with collision detection. Output: one self-contained document.
- **Validate + repair (quality).** Deterministic linting *rejects* broken specs loudly; for merely ambiguous ones, AI drafts fixes *into the config as a proposal a human freezes* — never an auto-mutation. Validation answers "is it correct?" (blocking); repair

answers "is it clear?" (advisory).

> **WHY THE MESSY FRONT DOOR DOESN'T COMPROMISE DETERMINISM**
> Three of the four stages are fully deterministic; only quality-repair uses judgement, and its output is frozen before anything consumes it. The fuzziness is quarantined at the entrance, so the pipeline behind it stays perfectly reproducible. Reverse-derived inputs (SDK/MCP) also carry a lower trust signal, since their contract was inferred rather than declared.

I.3 Tracing the example: artifact → IR → surfaces

The funnel above is the general rule; this is the specific path Lumen's `POST /projects` takes through it, hop by hop.

ACQUISITION — MESSY IN, CANONICAL OUT `SOURCE → ARTIFACT`

The resolver locates `openapi/lumen.yaml`, authenticates with the producer's GitHub App token, clones the whole tree (the spec uses relative `$ref`s), inlines those references into one self-contained document, detects OpenAPI 3.1, and pins the exact commit. After this, nothing downstream knows the source was private, fragmented, or remote.

IN (MESSY)
private repo · branch `main` · multi-file spec with relative `$ref`s · auth required

OUT (CANONICAL)
one resolved OpenAPI 3.1 document + `{ sha: a3f9c2…, hash: 7b1e… }`

IR CONSTRUCTION — THE NORMALIZED OPERATION `ARTIFACT → IR`

Once linting clears it, the operation becomes a stable, identified IR node. The `id` is assigned by identity, not file position, so a later spec edit or reorder doesn't break diffs or any override bound to it.

```
# IR fragment — the operation, normalized and given a stable id
operation:
  id: op_createProject          # durable — anchors diffs & overrides across spec edits
  resource: projects
  method: POST
  path: /projects
  input:
    name:     { type: string, required: true }
    team_id:  { type: string, required: true }
    template: { type: string, required: false }
  output:
    Project: { id: string, name: string, created_at: datetime }
  auth: bearer
  provenance: { sha: a3f9c2…, artifact_hash: 7b1e… }
```

CONFIG — DRAFTED, REVIEWED, FROZEN `IR + JUDGEMENT`

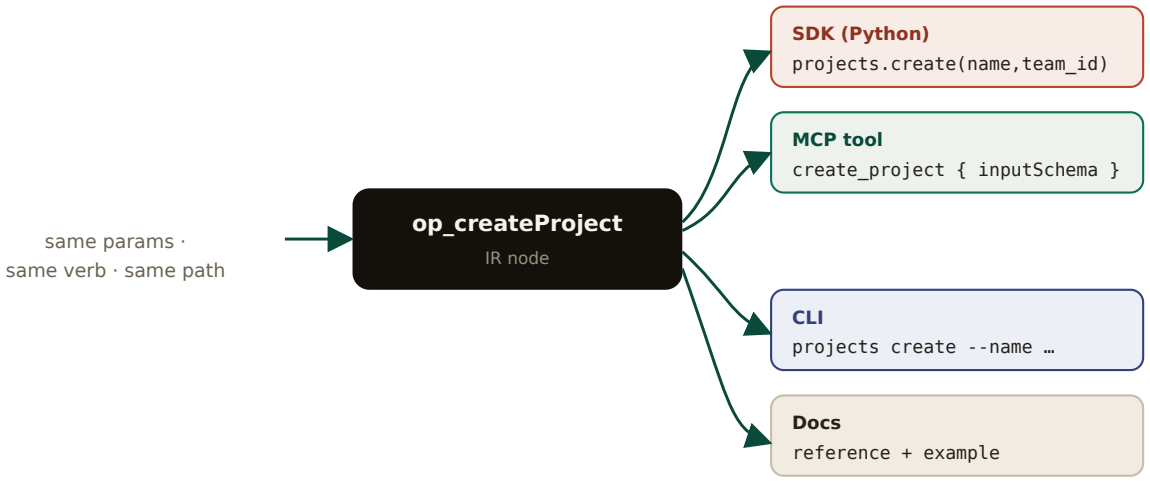
The platform drafts a sane config from the IR (the single place AI assists); the producer reviews and freezes it. All MCP curation lives here — which is how generation stays deterministic while tools stay well-described.

```
# frozen config — how the IR should render into each surface
mcp:
  op_createProject:
    tool: create_project
    description: "Create a new design project in a team. Returns the project id."
    scope: default            # included in an agent's default tool set
sdk:
  python:     { style: cursor_pagination, async: true }
  typescript: { style: cursor_pagination }
```

PROJECTION — ONE OPERATION, FOUR SURFACES `IR → SURFACES`

Projection renders the IR node into each surface. The clarifying view: the same capability wears four different clothes — the parameters, the verb, and the path are constant; only the shape around them changes.

FIG I.3 — One IR operation, projected four ways



Because all four derive from one node, they cannot drift out of sync — a spec change updates the IR once and every surface re-renders together. The MCP `inputSchema` is literally the IR input reshaped; the CLI flags are the same parameters; the SDK signature is the same contract.

```
# SDK (Python)
client.projects.create(name="Q3 brand refresh", team_id="t_88")

# MCP tool definition (advertised to the agent)
{ "name": "create_project",
  "inputSchema": { "type":"object", "required":["name","team_id"],
    "properties":{ "name":{…}, "team_id":{…}, "template":{…} } } }

# CLI
lumen projects create --name "Q3 brand refresh" --team-id t_88
```

The escape-hatch merge then re-applies any hand-written overrides on top of the freshly generated surfaces. `create_project` has none, so it passes through untouched — but the merge still runs, which is exactly what guarantees the next regeneration won't erase anyone's custom code.

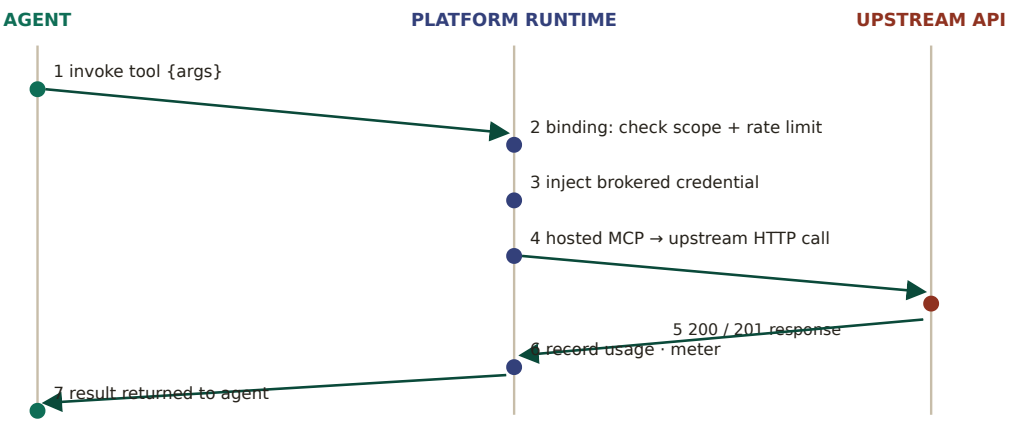
I.4 The stages, as a table

STAGE	INPUT	OUTPUT	GUARANTEE
Acquisition	Source (location × type)	Canonical artifact + pin	Origin-blind downstream
Ingestion	Canonical artifact	Validated artifact	Bad specs fail loud
IR build	Validated artifact	Content-hashed IR	Stable operation IDs
Config	IR + human review	Frozen config	Judgement captured once
Projection	IR + config	SDK · MCP · CLI · docs	Surfaces never drift
Merge	Surfaces + overrides	Final surfaces	Overrides survive regen
Host & register	Final surfaces	Versioned connector	Discoverable + isolated
Bind	Connector + consumer	Binding → live tool	Scoped + brokered
Runtime	Agent tool call	Result + usage record	No raw secrets cross
Loop	New source revision	New connector version	Breaking changes gated

I.5 Runtime — what happens on a live tool call

Generation is design-time; this is run-time — the path a single tool call takes once an agent is bound. It crosses three zones, and the platform sits in the middle brokering trust.

FIG I.4 — A live tool call, traced across zones



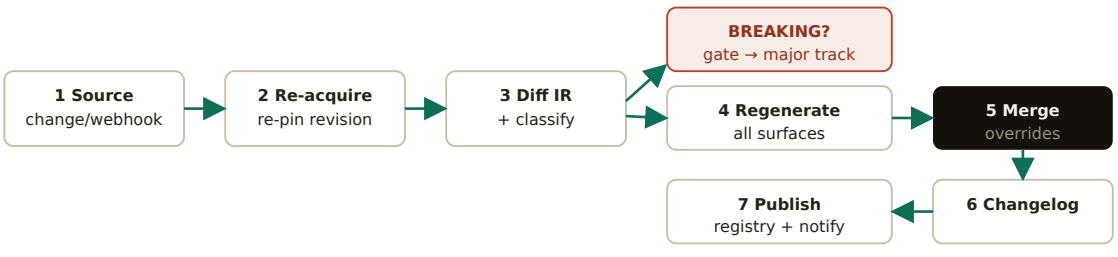
The agent never holds the producer's credentials and never talks to the upstream directly — the runtime enforces scope and rate limits, injects the brokered token into the sandboxed server, and meters the call on the way back for telemetry and settlement.

1. **Invoke** — the agent decides to call `create_project` and sends arguments to the binding endpoint.
2. **Govern** — the runtime checks the binding's scope and rate limits before anything else happens.
3. **Brokered auth** — it injects the consumer→producer token from the secrets broker into the sandboxed MCP server; neither party sees the other's raw secret.
4. **Upstream call** — the hosted server makes the real `POST /projects` request to Lumen.
5. **Response** — Lumen returns `201 { id }`.
6. **Meter** — the runtime writes a usage record (who, what, latency, status) feeding telemetry and settlement.
7. **Return** — the result flows back to the agent, which continues its reasoning.

I.6 The loop — when the spec changes, the pipeline re-runs itself

When a producer changes their spec, the pipeline re-runs automatically — and safely. This loop *is* the engine Part II's Auto-Sync operates, and its diff/classify step is what Preview Builds runs in dry-run.

FIG I.5 — Spec-driven regeneration loop with the breaking-change gate



Additive changes (new optional field/operation) classify MINOR and auto-publish; removals, type changes, and newly-required params classify BREAKING and are gated to a human + a new major track, so nothing silently breaks a bound agent.

Walking it with the example — Lumen adds an optional `description` parameter to `POST /projects` and pushes to `main`:

1. **Trigger** — the repo webhook fires; the platform learns the source changed.
2. **Re-acquire** — acquisition re-fetches and pins the new commit SHA.
3. **Diff & classify** — the new IR is diffed against the old; a new *optional* parameter is additive → classified **MINOR**.
4. **Gate** — minor changes auto-publish. Had `description` been *required*, or had a field been removed, it would classify **BREAKING** → blocked from auto-publish, routed to a new major track, dependents notified.
5. **Regenerate & merge** — all surfaces re-render from the new IR; escape-hatch overrides re-merge.
6. **Changelog & publish** — a human-readable diff is issued; the connector ships as `v1.3.0`; the registry updates.

Every bound agent keeps working without intervention — the existing `create_project` tool simply gains an optional field. Because regeneration is keyed by IR hash, re-running on an unchanged spec is a no-op and re-running on a changed spec is deterministic, so the loop can fire on every push without fear.

PART II

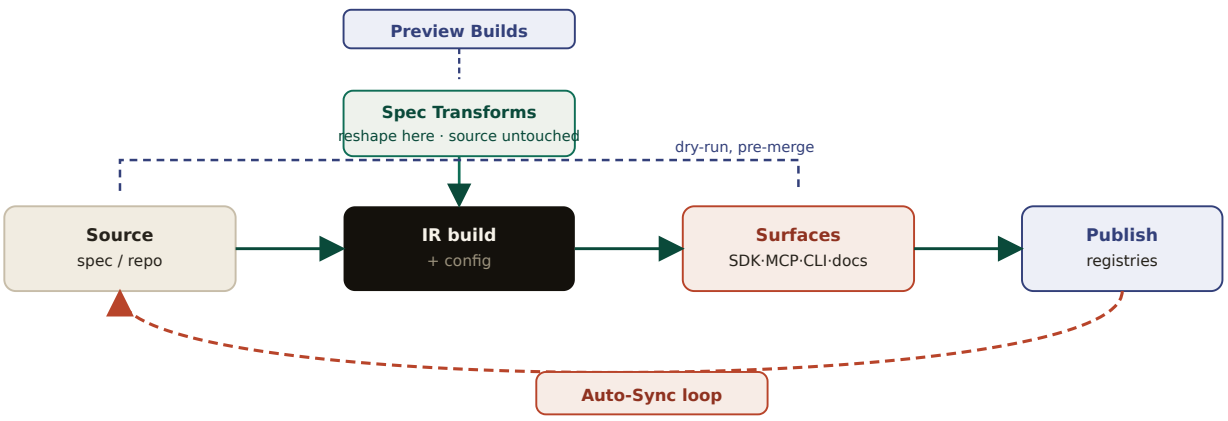
The Developer-Experience Layer

Three capabilities built on the Part I factory — keeping surfaces in sync automatically, reshaping messy specs without touching the source, and previewing a change's blast radius before merge. Adopting Stainless's best ideas while fixing its structural gaps.

II.0 What we are building, and where it sits

These three capabilities are **not** the factory — they sit on top of it as the developer-experience layer, turning "generates connectors" into "genuinely better than maintaining SDKs by hand or paying for a black box."

FIG II.1 — The three capabilities as interventions on the factory spine



Transforms intervene at IR construction (improving *all* surfaces); Preview taps the spine in dry-run before merge; Auto-Sync is the Part I regeneration loop run as a standing service.

Each capability is mostly assembled from machinery Part I already defines — which is why the three are cheap to build once the factory exists:

CAPABILITY	REUSES FROM PART I	ADDS
Spec Transforms	The config layer (I.3) + IR construction (I.2)	A declarative transform-rule engine applied at build time
Preview Builds	The diff + classify step of the loop (I.6), run in dry-run; projection (I.3)	A VCS app/action + PR-comment renderer + affected-consumer lookup
Auto-Sync	The entire regeneration loop (I.6) + escape-hatch merge	Standing trigger service + registry-publish + dependent notification

II.1 Stainless's disadvantages — and how we beat them

STAINLESS LIMITATION	HOW THE CONNECTOR NETWORK SOLVES IT
REST/OpenAPI-centric — weak for WebSockets, SSE, gRPC.	The projection-agnostic **IR** (Part I) means new protocols are a new adapter/projection, not a rebuild.
Black box & opinionated — you accept their SDK shape and pay for it.	Open IR + reviewable config + **escape hatch**. You own the model and output; overrides survive regeneration.
Not a full CI/CD pipeline — you still manage builds, versioning, publishing.	**Auto-Sync** owns the entire loop: regenerate → semver → breaking-change gate → publish to registries → notify dependents.
DSL lock-in — OpenAPI-as-source hits limits unless you migrate to their format.	Source stays **standard OpenAPI forever**. **Transforms** are config applied at build time — never a migration.
Managed-only — limited on-prem.	**Self-hostable**, decisive for regulated verticals (legal, finance) where data can't leave the perimeter.
SDK-centric DX — preview/transforms serve SDKs.	SDK, MCP, CLI, docs are **co-equal projections**; every DX feature serves all surfaces, including agent-facing MCP.
Vendor/continuity risk — recent coverage frames the Stainless SDK generator as winding down.	You own the artifacts, IR, and pipeline; every connector is reproducible from pinned inputs.

II.2 Capability — Auto-Sync / Maintenance Engine

IDEA

Keep every generated surface — SDKs across N languages, the MCP server, the CLI, the docs — automatically in lockstep with the upstream API contract, so a producer *never* hand-maintains multi-language SDKs. When the spec changes, all surfaces regenerate, re-version, and (when safe) republish, unattended.

The problem it kills. With hand-maintained SDKs, statically-typed languages (Go, Java, C#) need a code edit, review, and release for every field added — at scale, a team opening hundreds of reconciliation PRs a year. Auto-Sync makes the whole multi-language surface uniform and hands-off.

IN SCOPE

- Watch the pinned source for change (webhook or poll).
- Re-acquire → rebuild IR → diff → classify (patch / minor / breaking).
- Regenerate all surfaces; re-merge escape-hatch overrides.
- Auto-assign semver; auto-publish non-breaking to registries (npm, PyPI, …) + update the registry.
- Emit a changelog; notify dependent consumers/agents.

BOUNDARY — OUT OF SCOPE

- Never auto-publishes a *breaking* change — gated to a human + major track.
- Does not fix a broken spec (Transforms) or repair an invalid one (validation rejects it).
- Does not manage the producer's API or deploy their backend.
- Does not judge whether the change is *wise* — only that surfaces reflect it faithfully.
- Phase 1 language set bounded (Python, TypeScript; Go/Java later).

How it works. It is the Part I regeneration loop (FIG I.5) operating as a standing service. Safety rests on three inherited properties: regeneration **keyed by IR hash** (unchanged spec = no-op), the **breaking-change classifier** driving the gate, and the **override merge** that never loses hand-written code.

WHERE IT BEATS STAINLESS

Stainless automates SDK *builds* but stops at the door — you still own versioning and registry publishing. Auto-Sync owns the **full loop through publish**, adds **breaking-change gating** Stainless doesn't model, and syncs **every surface, not just SDKs** — including the MCP tools your agents depend on.

II.3 Capability — Spec Transforms

IDEA

A declarative way to reshape a messy real-world spec into a clean one *for generation only*, without ever editing the producer's source. Transforms live in the frozen config, apply deterministically on every build, and operate at the IR level so one fix improves *all* surfaces at once.

The problem it kills. Auto-generated specs are routinely wrong — an `account_id` typed as a number when it's a string, missing descriptions, ugly names — and you often can't get the producer to fix the source. Transforms correct the contract at build time and document why.

```
# a transform in the frozen config — original source file is never touched
transforms:
  - target: $.paths..parameters[?(@.name=='account_id')].schema
    op: set_type
    value: string
    reason: "upstream auto-gen types IDs as number; they are strings"
    temporary: true     # fail loudly once upstream fixes it, so we can delete the rule
```

IN SCOPE

- Declarative rules in config: target (JSONPath-style) + operation.
- Operations: set/override type, rename, add/remove field, set description, regroup resources, mark deprecated, merge/split operations, add examples.
- Applied deterministically every build; source stays pristine and standard.
- "Temporary fix" semantics — fail when upstream is fixed, prompting removal.
- Versioned with the config; effect visible in Preview Builds.

BOUNDARY — OUT OF SCOPE

- Deterministic *data* operations only — never arbitrary code (that's the escape hatch).
- Reshape the contract's *representation*; never change the API's runtime behavior.
- Not for net-new business logic or custom helpers.
- Not a permanent substitute for fixing the source — each rule carries a reason + ideally a sunset.

WHERE IT BEATS STAINLESS

Stainless transforms reshape the spec for *SDK* generation. Ours apply at the **IR layer**, so one corrected description flows into the SDK docstring, the **MCP tool description**, the CLI help, and the docs simultaneously. And rather than a proprietary DSL, our rules align with the open **OpenAPI Overlay Specification** (the same standard Speakeasy's overlays build on) — so the transform layer itself is portable and producers are never locked into our format.

II.4 Capability — Preview Builds

IDEA

Before a spec change is merged, show the contributor exactly how it ripples through every surface — diffs, the semver/breaking verdict, and which live consumers are affected — right inside their pull request. Turn "shipping API changes is scary" into "you see the blast radius before you commit."

The problem it kills. Without preview, engineers can't see how an OpenAPI edit affects generated surfaces until after it ships; the loop is long and risky, so teams avoid changing the API. Preview moves that feedback into the PR.

FIG II.2 — Preview build flow on a pull request

Nothing is published — the run is read-only and pre-merge. The release build reuses the last preview, so what you reviewed is what ships. The dry-run reuses Part I's diff + classify step.

IN SCOPE

- Triggered by a PR/MR that changes the source spec (GitHub Action / GitLab CI).
- Dry-run of the full pipeline: acquire PR spec → IR → diff vs current → project all surfaces.
- PR comment with per-surface diffs, semver/breaking verdict, suggested changelog, downloadable preview artifacts.
- Affected-consumer list (which bound agents touch the changed operations).
- Release build reuses the latest preview — no surprises at merge.

BOUNDARY — OUT OF SCOPE

- Never publishes — strictly read-only and pre-merge.
- Interactive preview requires a PR workflow; direct-to-main pushes get auto-builds without it.
- Phase 1 supports GitHub + GitLab; other VCS via generic CI later.
- Surfaces escape-hatch override conflicts but does not auto-resolve them.

WHERE IT BEATS STAINLESS

Stainless previews show *SDK* diffs. Ours show **every surface** plus the **breaking-change verdict inline** — tied to the gate Stainless lacks. We surface **MCP tool changes** (a renamed tool silently breaks live agents) and **which bound agents are affected** — a downstream view only possible because we hold the registry and binding graph.

II.5 Build order & dependencies

STEP	CAPABILITY	DEPENDS ON
1	Spec Transforms	IR build + config layer. Lowest-risk, immediately improves output quality, prerequisite for clean previews.
2	Preview Builds	IR diff + classifier + dry-run projection + a VCS app/action. Shows transforms' effect, so it follows them.
3	Auto-Sync	Everything above + registry publishing + the breaking-change gate. Preview's logic run unattended on merge.

SEQUENCING LOGIC

Transforms make output *clean*; Preview makes change *safe to see*; Auto-Sync makes maintenance *disappear*. They share one engine — the IR diff + classifier + projector from Part I — so this order means each reuses the last's machinery rather than duplicating it.

II.6 Global boundaries — what this plan does *not* cover

- The network layer (registry, identity, governance, settlement) — referenced only where a DX feature consumes it (e.g. Preview's affected-consumer list).
- Non-REST protocols — the IR permits them, but GraphQL/gRPC adapters and projections are a separate roadmap item.
- The producer/consumer portals' full UX — only the PR-comment and config surfaces touched by these features are in scope.
- Hosting-runtime internals (isolation, scaling) — covered by the Platform Build Plan; Part I covers only the runtime call path.

IN ONE SENTENCE

A source climbs once to a hashed IR and fans out into surfaces that cannot drift (Part I) — and on that foundation we build Stainless's three best ideas, so transforms improve every surface, previews show the full blast radius including agents, and Auto-Sync closes the whole loop through publish that Stainless leaves to you (Part II).

PART III

User Flow & Prior Art

What a user actually sees, clicks, and does at each stage — designed from how Fern, Stainless, and Speakeasy work today. The guiding constraint: user-friendly enough for a first-timer, but built for developer ergonomics first.

III.0 Knowledge base — how the comparable systems work

Before designing the flow, here is the distilled prior art. Three mature systems have converged on a remarkably similar shape, which tells us what developers actually expect. The convergence is the signal.

DIMENSION	FERN	STAINLESS	SPEAKEASY
Entry surface	CLI-first (`fern init`, `fern generate`)	Web Studio + CLI (`stl preview`)	Interactive CLI (`speakeasy quickstart`) + app
Config	`fern/` dir: `fern.config.json` + `generators.yml`	`stainless.yml` + OpenAPI spec	`.speakeasy/workflow.yaml` (sources → targets)
Local iteration	`generate --preview` / `--local` (no quota)	Language Server (in-editor) + `stl preview --watch` TUI	`speakeasy run` locally
Diagnostics	`fern check` validation	Error/Warning/Note panel, always outputs something	Spec lint + validate in `run`
Change workflow	PR opened against SDK repo on spec change	PR comment with diffs, diagnostics, install link; merge → build	GitHub Action opens PRs (daily / on change)
Non-destructive edits	OpenAPI overrides file	Config transforms	OpenAPI **Overlays** (open standard)
Output	Multi-repo; SDK, docs, CLI	Multi-repo (config / staging / production)	Multi-repo; SDK, MCP, CLI, Terraform, tests
Publish	Auto to npm, Homebrew, GitHub Releases	"Codeflow" auto-release to package managers	Auto to package registries
CI auth	Token	GitHub OIDC (no secret to rotate) or API key	API key / workspace
Protocols	OpenAPI, AsyncAPI, gRPC, OpenRPC	REST/OpenAPI only	OpenAPI-native

THE CONVERGENT PATTERN — WHAT "GOOD" LOOKS LIKE

- **Two surfaces, one config.** A visual Studio for onboarding and exploration, a CLI for power users and CI — both reading the same version-controlled config. Never force one or the other.
- **Config-as-code.** The spec + a config file live in the producer's repo. The config is the durable, reviewable, diff-able artifact — not hidden in a SaaS database.
- **Fast inner loop.** Local preview and a language server bring generation feedback into the editor (completions, inline validation, coverage hints) so the edit→see loop is seconds, not a round-trip to a build server. This is the single biggest ergonomics win, and it pairs naturally with AI coding tools.
- **Diagnostics-first, never hard-fail.** The generator always produces something and raises graded diagnostics (Error/Warning/Note) that guide a fix. A messy spec degrades gracefully; it doesn't bounce.
- **The PR is the review surface.** A spec change posts a PR comment with per-surface diffs, the version verdict, and install instructions; merge triggers the build. Generation lives alongside the code it produces.
- **Open overlays, not a proprietary DSL.** Non-destructive spec edits use the OpenAPI Overlay standard so nothing locks the producer in.

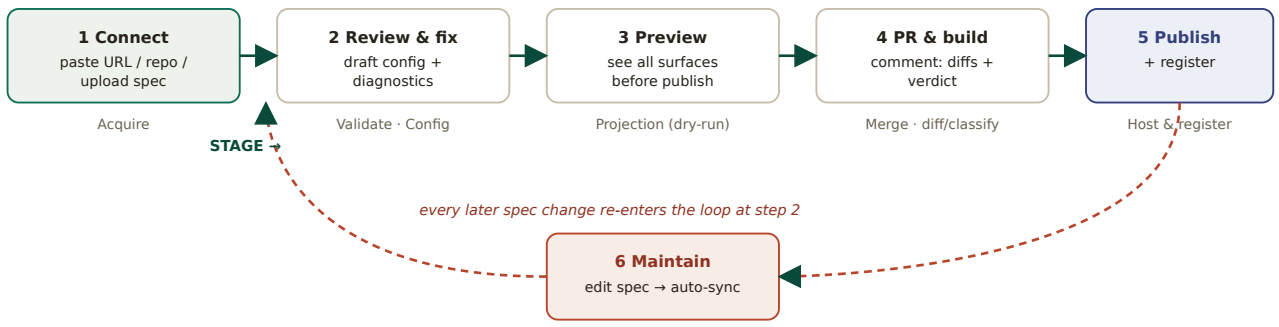
OUR STANCE

We adopt all six convergent patterns wholesale — they are table stakes, and reinventing them would only produce a worse version. We differentiate on the axes Part I and II already established: a protocol-agnostic IR, surfaces beyond SDK (MCP/CLI/docs as co-equals), the full loop through publish, and a self-hostable, open-format stack.

III.1 The producer journey — what they see, click, and do

A producer's path from "I have an API" to "my connector is live and stays in sync." Each step maps to a Part I stage, and offers the same job through either the Studio (visual) or the CLI (terminal) — the producer picks; the config is identical underneath.

FIG III.1 — The producer journey, mapped to pipeline stages



Steps 1–5 are the first-time path; step 6 is the steady state. Every subsequent spec change re-enters at step 2, so the journey is a loop, not a line.

1 · CONNECT A SOURCE `ACQUIRE`

Studio: click *New connector* → paste an OpenAPI URL, connect a GitHub repo (OAuth), or drag in a file. If the repo has several specs, a picker asks which. **CLI:** `cn init --openapi ./spec.yaml` (or a URL) scaffolds a version-controlled `cn/` folder with the config and a pinned source reference. Either way the resolver fetches, resolves `$ref`s, and pins the revision.

2 · REVIEW THE DRAFT & FIX THE MESS `VALIDATE · CONFIG`

This is where most time is spent, so it gets the richest UX. The platform has already *drafted* a config (resource grouping, tool names and descriptions) and surfaced graded diagnostics. The producer:

- **In the Studio** — sees three panes: the spec, the config, and a live preview of the generated surfaces; clicks a diagnostic (Error/Warning/Note) for a guided fix; applies an **Overlay** to correct a messy type or name without touching the source.
- **In their own editor** — runs the **language server** (VS Code, Cursor, Neovim) for the same completions, inline validation, go-to-definition between config and spec, and coverage hints — and can lean on AI coding tools right alongside it. This is the fast inner loop.

3 · PREVIEW EVERY SURFACE `PROJECTION (DRY-RUN)`

CLI: `cn preview` opens a TUI showing build progress, diagnostics, and links; `cn preview --watch` regenerates on every save. The producer sees the actual SDK shape, the MCP tool list, the CLI commands, example snippets, and type definitions — and can install the preview artifacts to test locally — all before anything is published. Local preview has no quota; only cloud builds are rate-limited.

4 · OPEN A PR & BUILD `MERGE · DIFF/CLASSIFY`

The producer commits the spec/config change as a pull request. The platform's check posts a **PR comment**: per-surface diffs (SDK methods, MCP tools, CLI commands), the semver/breaking verdict, the list of affected consumers, and install instructions for the preview build. Escape-hatch overrides are re-merged. The reviewer sees the full blast radius inline.

5 · PUBLISH & REGISTER HOST & REGISTER

On merge, the connector builds at its computed version, surfaces auto-publish to their registries (npm, PyPI, Homebrew), each SDK lands in its own repo, the MCP server deploys to the runtime, and the connector appears in the registry with its docs and trust signal. CI auth uses OIDC, so there is no secret to rotate.

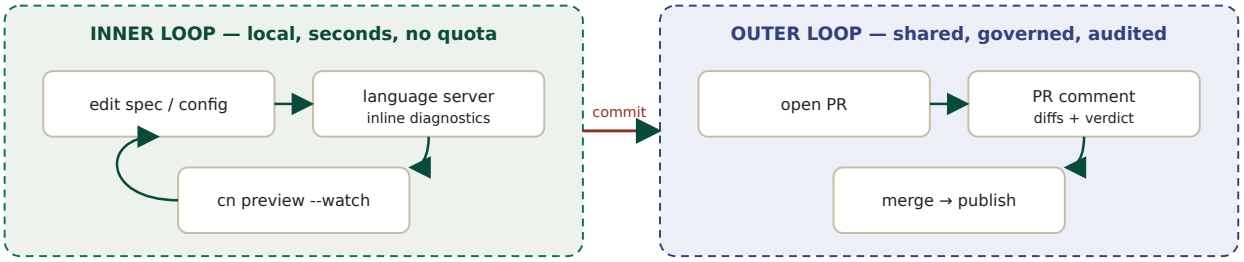
6 · MAINTAIN — IT STAYS IN SYNC BY ITSELF LOOP / AUTO-SYNC

From here the producer just edits their spec. Every push re-enters at step 2: diff, classify, regenerate, PR comment, merge, publish. Non-breaking changes flow through unattended; breaking ones are gated to a major version with dependents notified. The producer never hand-maintains a single SDK.

III.2 Inner loop vs outer loop — the two cadences

The ergonomics hinge on separating a fast, local, private *inner loop* from a governed, shared *outer loop*. Conflating them is what makes tooling feel slow; keeping them distinct is what makes it feel fast *and* safe.

FIG III.2 — The two loops a producer lives in



The inner loop is the producer alone, locally, iterating in seconds with no network round-trip. The outer loop is the team: a PR, a review of the blast radius, a governed publish. The same config drives both.

III.3 The consumer journey — discover to bound

The consumer side is deliberately lighter — most of the work was done by the producer and the platform. A consumer (or their agent) goes from finding a capability to using it in a few clicks.

1. **Discover** — browse or search the registry by capability; or the agent queries it directly. The connector page shows capabilities, available surfaces, versions, trust signal, and interactive docs.
2. **Try** — an in-browser API Explorer runs real requests with handled auth, and shows language-specific snippets — no install needed to evaluate.
3. **Take the surface** — for developers, `npm install` / `pip install` the SDK or grab the CLI; for agents, click *Add to agent* to bind the MCP surface.
4. **Bind** — one action creates a binding: scoped tools, brokered credentials, an endpoint handed to the agent. The agent gains the tools immediately; revoking removes them.
5. **Monitor** — a usage dashboard shows calls, latency, errors, and spend for the consumer's own slice.

AGENT-READY BY CONSTRUCTION

Because every connector ships an MCP surface and machine-readable metadata (per-command schemas, a drop-in context file, hardened input validation), an agent can discover, reason about, and call a capability with the same artifacts a human developer uses — the consumer journey works for software and people alike.

III.4 Where our flow goes beyond theirs

IN THE COMPARABLE TOOLS	IN OUR FLOW
Preview shows **SDK** diffs.	Preview shows **every surface** (SDK, MCP, CLI, docs) plus the breaking-change verdict and the affected-consumer list inline.
Output is SDKs (some now add MCP/CLI).	SDK, MCP, CLI, and docs are **co-equal projections** of one IR from day one — none is an afterthought.
The flow ends at the producer's published package.	The flow continues into **discovery and binding** — a consumer or agent can use the capability without the producer doing integration work.
REST/OpenAPI in; SDK out.	Any source **including an existing SDK or MCP server** climbs to the IR; the journey is identical regardless of input.
Managed service.	Same flow runs **self-hosted**, so the spec and data can stay inside a regulated perimeter.

IN ONE SENTENCE

We keep the inner-loop speed and PR-centric, config-as-code ergonomics that developers already expect from Fern, Stainless, and Speakeasy — then extend the journey at both ends: any input climbs in, and every surface (plus discovery and binding) comes out.